

OpenGL Capture/Replay: Deep Developer Guide

Updated: 2026-09-28 16:05:48

Audience: developers extending or debugging the `gl_capture` pipeline. This guide is intentionally implementation-level and maps behavior to exact files and function entry points.

A. System intent and invariants

- Capture runs **inside target process** (hook DLL) and writes a byte-for-byte call stream that replay can consume out-of-process.
- Replay reconstructs behavior by **decoding each record in order**, remapping object names, and issuing real GL calls on a fresh context.
- For any custom API: capture payload schema, replay decoder, and formatter output must evolve together.
- Frame boundaries are explicit records, not inferred from API heuristics.
- Safety-first rule: malformed pointer payloads are skipped rather than passed as potentially invalid pointers to the driver.

B. End-to-end call path (with ownership)

Phase	Runtime action	Where in code	Developer notes
1. Launch prep	UI reads CLI/named args, validates paths, chooses capture mode (all vs oneframe), sets env vars used by hook DLL.	ui/main.cpp ReadLaunchOptions() LauncherFrame::StartCapture()	If launch options behave oddly, debug <i>raw Windows argv</i> path, not wx parser assumptions.
2. Suspended process start	Target process created with <code>CREATE_SUSPENDED</code> so hook can patch imports before first GL call executes.	ui/main.cpp CreateProcessA + ResumeThread path	Any failure before ResumeThread means target never executed app code.
3. DLL injection	Launcher writes DLL path to remote memory and starts remote thread at LoadLibraryA.	ui/main.cpp InjectHookDll()	Injection success is based on remote LoadLibrary return value.
4. Hook attach	On <code>DLL_PROCESS_ATTACH</code> : open trace, configure mode, optionally open oneframe event, patch imports of all loaded modules.	interceptor/dllmain.cpp DllMain()	Patch scope is broader than EXE to handle framework/helper DLL GL calls.
5. Static + dynamic interception	IAT patch redirects GL/WGL/GDI and loader APIs; MyGetProcAddress/MyWglGetProcAddress return wrappers for dynamically loaded symbols.	interceptor/dllmain.cpp PatchModuleGLHooks() MyGetProcAddress() MyWglGetProcAddress()	Critical for loaders that resolve all symbols at runtime.
6. Capture serialization	Wrapper begins call, appends scalar/blob fields, ends call to flush record. SwapBuffers/wglSwapBuffers emit frame marker.	common/trace_writer.h interceptor/custom_wrappers.cpp generated/gl_wrappers_generated.cpp	Custom wrappers own byte layout decisions for pointer/query APIs.
7. Replay decode	Replay UI reads trace file, builds call list lines, dispatches each non-frame record by GLFuncId.	replay/main.cpp generated/gl_replay_decode.cpp	Formatters affect diagnosis UX only; replay handlers affect behavior.
8. Real call issue	Replay handlers map IDs, rebuild pointer data as needed, then call <code>g_real.*</code> functions.	replay/custom_replay.cpp generated/replayers	Malformed payload -> skip unsafe call.

C. Binary trace contract (byte-level)

Layer	Layout	Definition	Where read/written
File header	8 bytes magic GLCAP001	Trace identity / compatibility gate.	common/trace_format.h TraceWriter::Open() replay/main.cpp loader
Per-record header	uint16 func_id uint16 reserved uint32 data_len	func_id is GLFuncId, or frame marker 0xFFFF. reserved may carry typed flag.	common/trace_format.h TraceWriter::WriteRecordUnlocked()
Frame boundary	func_id=0xFFFF,data_len=0	Explicit frame delimiter emitted at buffer swap interception.	TraceWriter::MarkFrameEnd() replay/main.cpp frame parsing
Fixed pointer payload (custom pattern)	[u32 byteLen][blob bytes][u64 originalPtr]	Used by many pointer-vector/matrix wrappers for deterministic replay + richer call list display.	interceptor/custom_wrappers.cpp: WriteFixedPointer() replay/custom_replay.cpp replay/custom_format.cpp
Query payload (custom pattern)	[GLenum pname][u32 count][values][u64 outputPtr]	Captures queried values and caller output pointer for diagnostics.	glGetBooleantv/glGetFloatv/glGetIntegerv wrappers + replay/formatter
Typed call extension	reserved = kTypedRecordFlag and typed headers	Versioned typed schema present for extensibility while preserving base GLCAP001 records.	common/typed_trace.h TraceWriter::BeginTypedCall/EndCall

D. File-by-file ownership index (fast “where do I edit?” map)

Problem / task	Edit here first	Companion files	Why
Add support for a new pointer API	codegen/functions.json (custom=true) interceptor/custom_wrappers.cpp	replay/custom_replay.cpp replay/custom_format.cpp codegen/generate.py outputs	Generated scalar path cannot infer arbitrary pointer payload size/meaning.
Call list line is unreadable / missing details	replay/custom_format.cpp	generated/gl_format_calls.cpp for non-custom APIs	Formatters are UI-only string builders; they do not affect replay correctness.
Replay crashes in driver from pointer call	replay/custom_replay.cpp	interceptor/custom_wrappers.cpp payload writer	Most driver crashes come from null/invalid pointer decode or count mismatch.
Heap corruption during capture in target app	interceptor/custom_wrappers.cpp QueryValueCount()	API-specific wrapper implementations	Wrong element count lets real driver overrun stack/heap of local capture buffer.
Function not intercepted at all	interceptor/dllmain.cpp PatchModuleGLHooks()/MyGetProcAddress() dress()	generated/gl_lookup_table.*	Could be dynamic symbol resolution or missing wrapper lookup entry.
CLI launch option not honored	ui/main.cpp ReadLaunchOptions()	LauncherFrame ctor defaulting	Raw CommandLineToArgvW parser is authoritative input parser.
Object IDs mismatch in replay	replay/custom_replay.cpp (Replay_glGen*/Delete*/Bind* paths)	generated/gl_replay_decode.cpp dispatch	Replay process has different real IDs; remapping must be complete.
Need to change core record format	common/trace_format.h common/trace_writer.h	replay/main.cpp readers custom replay/format decoders	Any structural format change is cross-cutting and high-risk.

E. Capture-side internals (interceptor)

E1) Import patching and symbol interception

- PatchImports is applied to **opengl32.dll**, **gdi32.dll**, and loader DLL imports (kernel32/KernelBase/api-ms-win loader shims) so both static imports and dynamic resolution paths are intercepted.
- **MyGetProcAddress** is the critical fallback hook for frameworks that resolve every GL entry point at runtime and never statically import it.
- **MyWglGetProcAddress** stores real function pointers and falls back to GetProcAddress(opengl32.dll, name) for GL 1.1 core exports that wglGetProcAddress may legally return null for.

E2) Client-memory arrays and draw capture strategy

Concept	Implementation	Important edge case
Array pointer origin	client_array_state.h tracks whether pointer was set while ARRAY_BUFFER bound (offset) vs no VBO (client pointer).	Interpretation is latched at pointer-setup call time, not draw time.
Byte-length computation	ClientArrayByteLength(desc, first, count, fixedSize) computes reachable bytes using GL stride rules.	Wrong size over-reads app memory or truncates replay data.
Indexed draws	Capture scans index data to get max index then snapshots reachable client arrays.	Element-array binding may be VAO-owned and not fully shadowable; code prefers querying real driver state when possible.
Interleaved arrays	Current wrapper stores format/stride/pointer-presence metadata only.	Not full-fidelity client memory snapshot for all interleaved layouts.

F. Replay-side internals

Area	Core functions/classes	Behavior contract
Dispatcher	generated/gl_replay_decode.cpp ReplayDispatch(...) switch	Every GLFuncId must route to a valid replay handler (generated or custom).
Custom decode safety	replay/custom_replay.cpp ReadFixedPointer(), ReplayQuery()	Bounds-check before reading; skip invalid payloads; avoid invalid pointer pass-through.
ID remapping	Replay_glGen*/Delete*/Bind* families	Captured IDs are logical; replay real IDs differ and must be translated consistently.
Call list formatting	generated/gl_format_calls.cpp replay/custom_format.cpp	Formatter must decode same schema as replay path for trustworthy diagnostics.
Frame stepping UX	replay/main.cpp CallListPanel / ReplayFrame::RunToIndex	Double-click/selection drives incremental replay to chosen call index.

G. Function-class matrix (capture strategy by API type)

API class	Typical examples	Capture strategy	Replay strategy	Formatter target
Pure scalar/value	glEnable, glViewport, glBlendFunc	Generated wrapper writes scalar args in order.	Generated decoder reads and calls g_real directly.	Generated call formatter.
Fixed-size input pointer	glVertex3fv, glColor4fv, glLoadMatrixf/d	Custom: write byteLen + raw bytes + ptr.	Custom: validate exact expected size then call real API.	Show ptr + typed values preview.
Variable-size query output	glGetBooleanv, glGetFloatv, glGetIntegerv	Custom: compute count from pname map, call real into local vector, write pname/count/values/ptr.	Decode captured values; replay has no caller output sink so values are local diagnostic/state only.	Show pname, ptr, count, values.
Object create/delete	glGenBuffers, glCreateTextures, glDelete*	Capture IDs from target process.	Recreate and map IDs; unmap on delete.	Show id arrays and remap-sensitive fields.
Client-memory draw arrays	glDrawArrays + pointer arrays, glDrawElements client index path	Snapshot reachable client-array bytes at draw point.	Bind replay-owned decoded array memory before issuing draw.	Summarize attached client-array snapshots/bytes.

H. Build/regenerate/test workflow (authoritative loop)

- **Regenerate:** cd gl_capture\codegen && python generate.py
- **Build:** cd gl_capture\build && cmake --build . --config Debug --target gl_capture_hook gl_replay gl_capture_ui --parallel 4
- **Capture:** gl_capture_ui -e <exe> -i "<args>" -d <run_dir> -o <trace>
- **Replay:** set GLCAP_REPLAY_AUTOEXIT=1 && gl_replay <trace>

I. Debugging playbooks

Symptom	Likely root cause	First three checks
Target crashes during capture (heap corruption)	Incorrect query/pointer element count in capture wrapper.	1) Inspect QueryValueCount/size mapping. 2) Confirm wrapper count logic for specific pname. 3) Ensure local vector size equals driver-written element count.
Replay crash in driver	Malformed pointer payload decoded as valid pointer/size.	1) Validate byteLen vs expected bytes. 2) Confirm null handling. 3) Guard short payload reads with remaining checks.
Call list says present/bytes but no semantic values	Formatter still using generic fixed-pointer summary path.	1) Add type-aware formatter function. 2) Decode leading scalar fields before pointer blob. 3) Emit bounded values preview.
No calls captured	Module doing runtime resolution path not patched or capture started too late.	1) Verify injection succeeded pre-resume. 2) Confirm loader hooks active. 3) Confirm traced module import patch pass ran for all loaded modules.

J. New-function implementation checklist (copy/paste for PRs)

- Mark function metadata in codegen/functions.json (custom when needed).
- Define exact payload schema in capture wrapper and document field order in comments.
- Implement replay decoder with strict bounds checking and graceful skip on malformed data.
- Add/extend formatter output with semantic fields + bounded value preview.
- Regenerate codegen outputs; do not hand-edit generated files.
- Rebuild Debug targets and run capture+replay on workload that exercises new function.
- Verify call list line includes useful semantic data (enums/count/ptr/values) and no text overflow.

Appendix: Frequently touched files

Path	Primary reason to open it
ui/main.cpp	Launch/inject pipeline, command-line options, capture-mode env wiring.
interceptor/dllmain.cpp	IAT patching strategy, GetProcAddress hooks, frame marker injection.
interceptor/custom_wrappers.cpp	Custom serialization logic for pointer/query/client-array APIs.
interceptor/client_array_state.h	Client-backed array bookkeeping and byte-range math.

Path	Primary reason to open it
common/trace_format.h	Trace file structural contract and frame marker constants.
common/trace_writer.h	Record writing lifecycle and oneframe behavior.
replay/main.cpp	Trace read loop, call-list UI, run-to-index behavior.
replay/custom_replay.cpp	Custom decode + safe real GL issuing path.
replay/custom_format.cpp	High-value diagnostics for pointer/query calls.
generated/gl_replay_decode.cpp	Dispatch map from function ID to replay handler.
generated/gl_format_calls.cpp	Generated value-argument call text.
codegen/generate.py	Emitter logic for IDs, wrappers, decode dispatch, formatters.
codegen/functions.json	Function metadata and custom flags; first stop for support changes.